

w10L54 Hierarchical Reinforcement Learning

For scaling a problem - we did functional approximation

Expect some structure in policy and value function approximation

Look at higher order structure

- Subproblems you solve while solving larger problem (repeated moves)
- Give direction to Central Station:
 - Which mode - want to minimize cost
 - I am dropped from sky here - Go to the main gate and take right,
 - Go to Kasturi baug, go to station, buy ticket, board train,
 - Humans are good at breaking the problem down

1st Aspect

Temporal Abstractions

- Abstract away someone notion of time
- Each of these will take different amount of time

2. How do I go to main gate:

- What does he mean by get down?
- Walk down of building, take left, left
- Can further break it down
- How do i walk to door (learnt behaviour)

You learn solutions, which can be used to solve problems

If you are solving a problem once in a while - you may not want to optimize it (how to reach station - once a semester

Effort I can save, use it somewhere else

How much do humans optimize solutions?

OCD - High Achievers have some degrees of OCD:

- You keep doing again and again
- Not necessarily doing perfectly

Optimization you do is not visible - **Cuban cigar rollers** ->

- How long does it take for someone to roll a cigar
 - Min or less than a min
 - Can do in less than 7 seconds (doing for 20+ years)
 - Humans optimize
 - In sports we see -> 14hrs a day for 30 year, with exact same precision

We quite often solve problem once

2nd Aspect:

Transfer / Reusability

- Something from what you know you can transfer

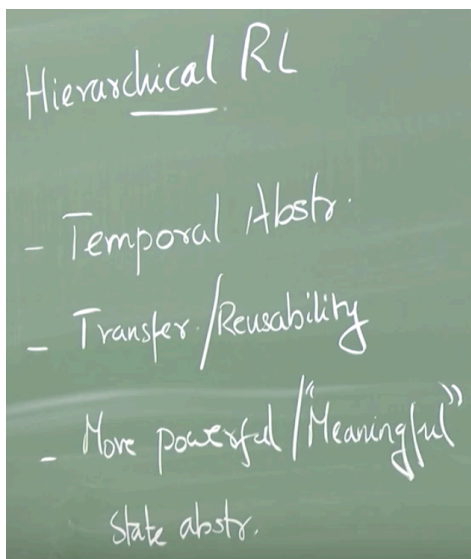
Disconnect between Planning community

- Given start and goal position
- Can apply operators if the pre-condition is satisfied
- Put phone if table is empty
- Post condition - table is not empty
- Already some operator was feasible earlier and now
- Some operators can become feasible now

Why learn at all - just do planning and be happy?

Learn solutions for sub operation so next time i use it

For solving different problem - stacking - will be putting objects



3rd Aspect:

More powerful / "meaningful" state abstraction

State abstraction -> fi-s

Optimum solution to going to main doesn't care where I am going after that

- It is the same solution

I can start focusing only on

1. Reduced state space -> going to station state space is large - mind blowing

- You will fail
- So many various variable - where to stand on platform
- Can ignore them, when I am getting out of the building
 - What is needed to solve at that point
 - Meaningful representation

2. Transfer:

- This is what really enables transfer
- Can't use central state var to other place
- Can use if it is defined in terms of this building

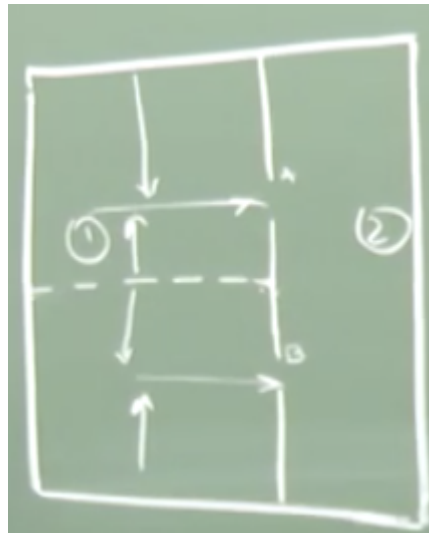
Look at different Hierarchical Framework:

- From 1996 -> I am working on it!
- Can take whole semester

How do we split the problem:

- Split so that you can use it for other problem
- In AI - how to identify hierarchy - no good answer - domain experts will tell
- Points of inflection - funnel -> all enter through main gain
- How do i get to funnel state -> i told you by knowledge of the building
- Geometric navigation problems are easy
- Fin Funnel and Bottleneck state
- Atari has significant dynamics so it is muddy

w10L55 Types of Optimality



this is policy

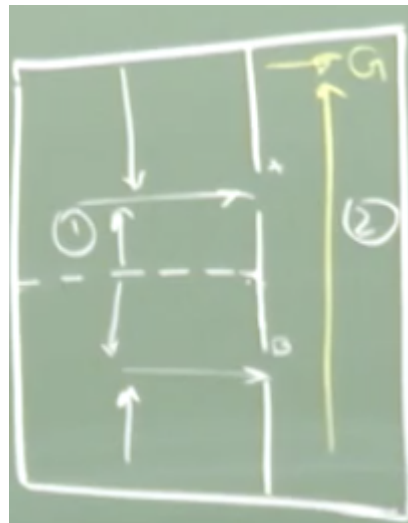
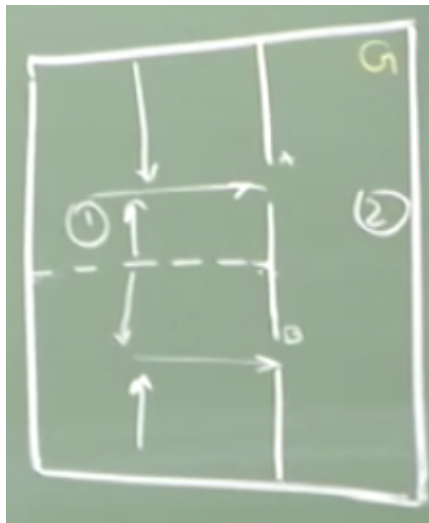
want to go from room1 to room2, which one is best way to go

Can start from anywhere

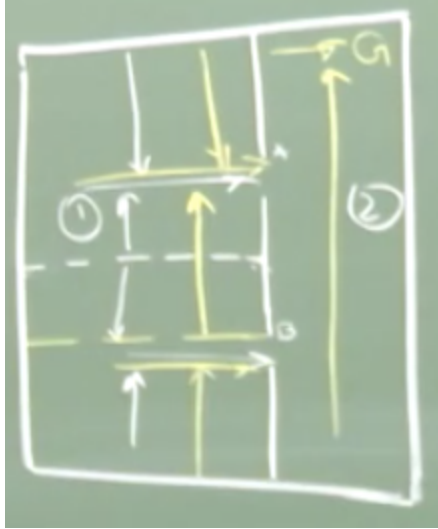
Give me policy to go to room 2

- Pick the closest door

My final goal is here: will policy change - or will be same. Ans doesn't depend on reward function



Policy in room2 go up



it will change as you may try to go up in room1

Is Yellow optimal?

Both Yellow and White are optimal - depends on how you define **optimality**

Hierarchically Optimal:

- You have to search through policies - that respects this hierarchy

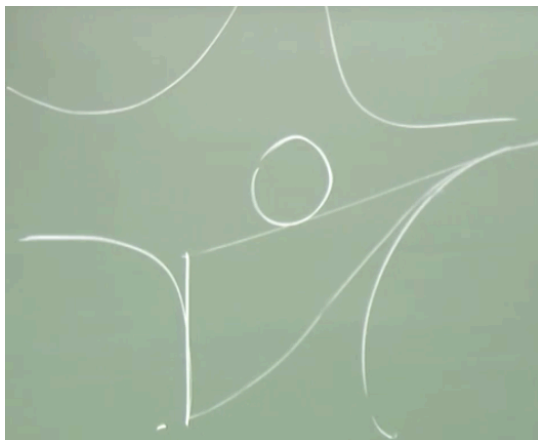
Structure - 1st go to room2

Asking how solving optimal sub-problem - not overall problem

- Could have used white policy - which is not optimal

Hierarchical ly optimal:

- Respect Hierarchy
- Yellow in room1 and yellow in room2
- It is optimal solution



going to mess from GC

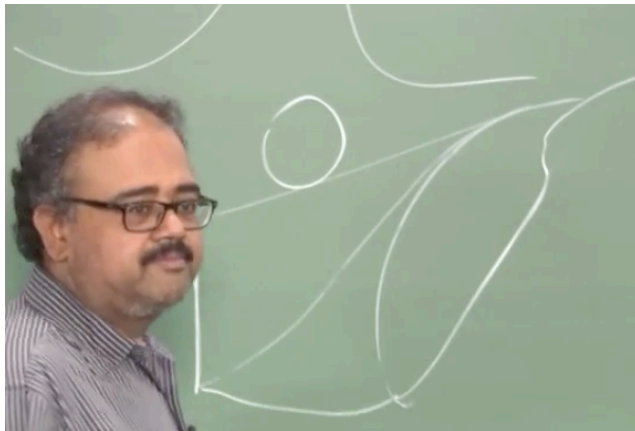
2. Recursively Optimal

- Put optimal sub-policies
- Respect hierarchy and each subproblem should be optimal

3. Flat Optimality

Find most basic

- **Gives more optimal solution**, which in turn gives better than recursive optimality
- Which is better solution to have?
- You have to give up on hierarchy and its benefits
- **Hard to Achieve (Assignment question)**



Gajendra Circle

1. Lower curve - Flat optimal - Globally optimal
2. Diagonal - Hierarchically optimal
3. Up and diagonal - recursively optimal

Hierarchical optimality or Recursively optimal, which is better

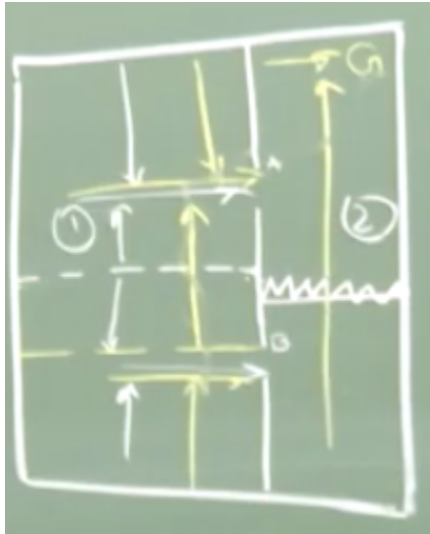
Recursive optimal is great!

- Easier to transfer
- I still have to solve whole problem - by treating solutions to sub problems as action
- Need the infrastructure
- I learned weird policy
- To learn yellow policy - I don't have worry about TD error propagating back
- Reward at goal has to propagate back

Hierarchical policy:

- Recursively policy is bad - optimal subsolution is bad

You have curved laser beam



Even with laser - White policy is still recursively optimal

Want to do hierarchical policy

In room2 bottom - you have only yellow policy

w10L56 Semi-MDP

Recursively optimality:

Have mechanism by which you can treat solution you have as action

- So learning horizon becomes smaller - don't have to take many-many action

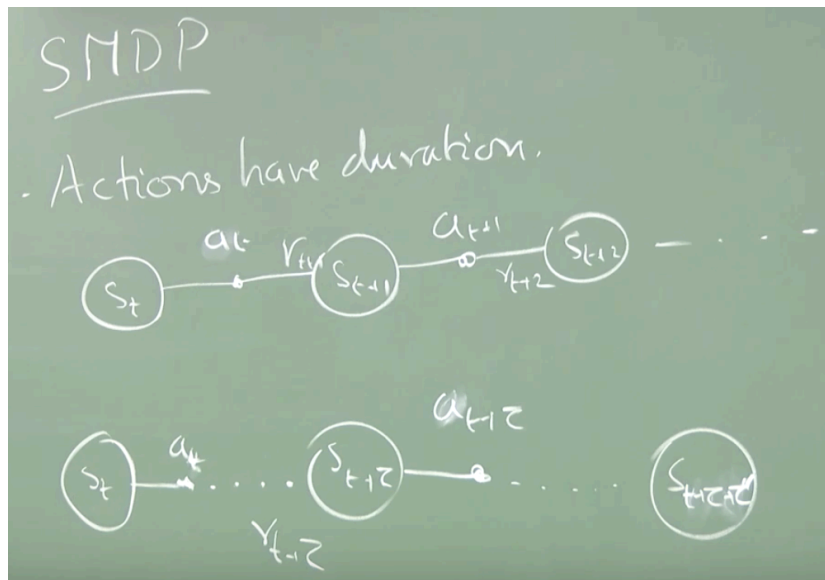
Foundational Structure is Semi-MDP

If depends on two previous states - it is second order Markov

In Semi-MDP:

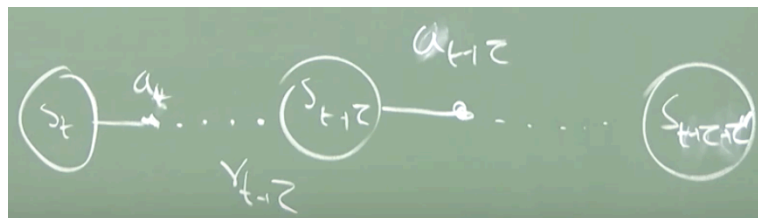
- Actions Have Duration

In MDP:



1st state diagram - MDP

In **SMDP**: some time elapses, find in next state



Tau + some other Tau

Next time - time taken will be different

- That is how will discount the reward
- No sub problems here
- Optimality - where the action takes but also how long it takes
- Tau is random variable = amount of time

Holding Time

- At time t , decide to take action but don't take till $\tau-t$
- I may be sensing the market

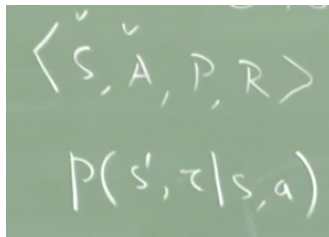
To determine next state to occur

- Completely NON-markov
- When will it show up - for brief period time dependence

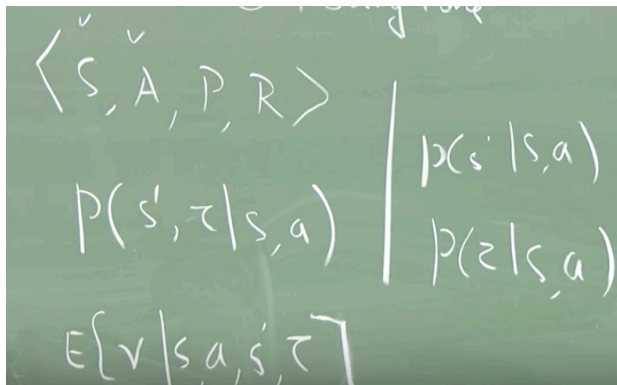
What happens after I take the action - transition time acceptable for RL folks

SMDP: doesn't tell what happens during those tau seconds

Eventually you will end up with same thing


$$\langle S, A, P, R \rangle$$
$$P(s', \tau | s, a)$$

we know S and A


$$\langle S, A, P, R \rangle$$
$$P(s', \tau | s, a) \quad \left| \quad \begin{array}{l} P(s' | s, a) \\ P(\tau | s, a) \end{array} \right.$$
$$E[v | s, a, s', \tau]$$

reward based on holding time

Classical way to write as two different functions - on right side of line (one on s and other on s')

RL people do joint distribution as shown in last eq.

Can write SMDP value function, Bellman equation

Recursive version of value function

Gamma - has to come from domain expert

Lookup ->

Lot of Hierarchical Frameworks has

REF: Survey paper : Recent advances in Hierarchical RL

- better than - original papers
- Author - fantastic write

SMDP Q-learning:

SMDP Q-Learning.

$$Q(s_t, a) = Q(s_t, a) + \alpha \left[\bar{r}_{t+\tau} + \gamma \max_{a'} Q(s_{t+\tau}, a') - Q(s_t, a) \right]$$

$$\bar{r}_{t+\tau} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots + \gamma^{\tau-1} r_{t+\tau}$$

If tau is 1 do gamma

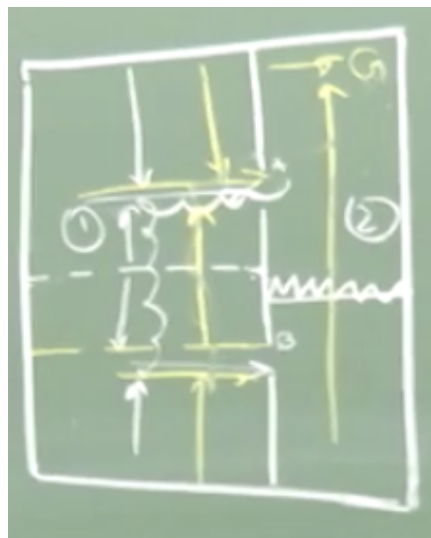
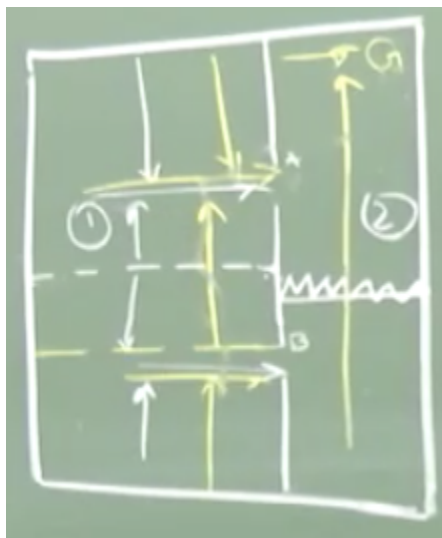
Hidden some stuff in gamma t+tau - what is r-t+tau -> sample over tau time step

Don't tell r-t+tau

Rate function - r -> accumulate tau time steps

$$\bar{r}_{t+\tau} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots + \gamma^{\tau-1} r_{t+\tau}$$

We are interested in:



reward curved line in room 1 going to room2 (right diagram)

Value of going to room2 will going to starting point at lower point

Reward fn: 1 when you reach goal, gamma discount along the way

SMDP

- doesn't look into the structure of (st, at)

at = go to room2

It is not simple action

- Temporal extraction

0.9 prob action will succeed

At the end of the day you want to optimize on discounted return

w10L57 Options Framework

Reference:

[R. S. Sutton, D. Precup, S. Singh. 1999. Between MDPs and semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning](#)

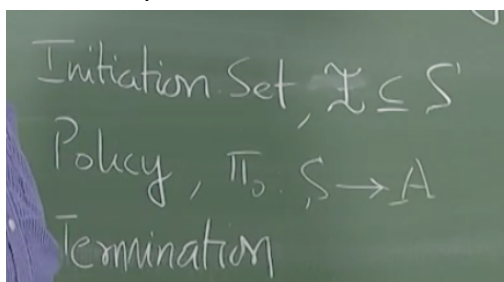
Last class looked at intuitive level and mathematical

Options Framework:

- Simplest of all
- Take solutions to sub-problem
- Encapsulate them as single action
- **Macro Action** (in planning community) put together actions - and call macro action
 - Climb down the stairs - single action
 - Slipper steps -> skip ahead some steps
 - Need to regain balance
- We live in stochastic world
- What do we need to specify for Macro Action
 - You need to specify Policy -> **Encapsulated Policy**
- How did i start the macro step -> where they are applicable (e.g. stand in front of the steps)
- Even for simple action - where to start and where to end (after 120 steps)

1. **Initiation**
2. **Policy to use while executing Macro action**
3. **Termination**

Classical options framework



policy - mapping of history of things happened - since i started doing this to next action
What were last action, last 2 or last 3 actions

Initiation Set, $\mathcal{I}_0 \subseteq \mathcal{S}$
 Policy, π_0
 Termination, $\beta_0: \mathcal{S} \rightarrow [0,1]$

① Option, $O = (\mathcal{I}_0, \pi_0, \beta_0)$
 Macro Actions.
 Encapsulated Policy
 Initiation Set, $\mathcal{I}_0 \subseteq \mathcal{S}$
 Policy, π_0
 Termination, $\beta_0: \mathcal{S} \rightarrow [0,1]$

In original - beat was probability

I am just defining the dynamics - it is not RL problem - Added o for options

Probability definition for Option $\rightarrow$ in **practice** make it 0 (does not terminate) or 1 (terminate)

- Indicator functions

Two Options:

1. Markov Options:

π_0 will be defined for current state

- Markov Options.
 - π_0 depends only on current state

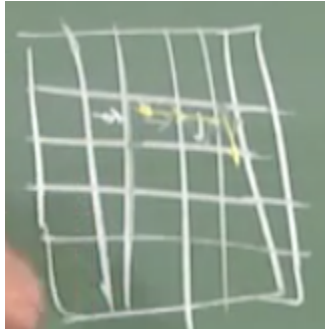
2. Semi-Markov Options:

- Sequence of states since the option was initiated
- π_0 depends on history - since the option started

- Semi-Markov Options
 - π_0 depends on history since the option started



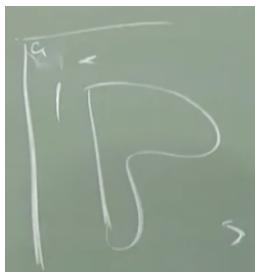
Start here (1st square) go right 2 steps, down 1 step



where i started I take action

- start from second square and then go two states right and one state down

For Markov - i should be taking same action



have two bottlenecks before reaching the goal

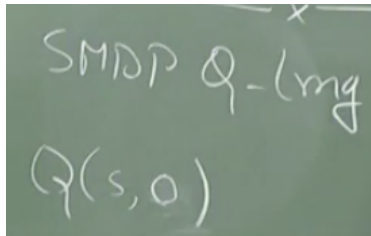
Just define options

- Which make it more exploration - move away start doing random walk
- Can explore lot more state space

w10L58 Learning with Option

How do you learn?

SMDP Q_Learning

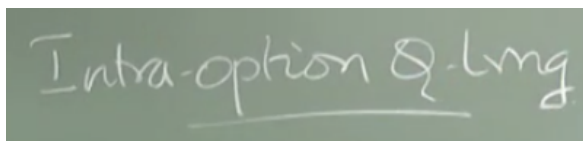


Take action part is clear - normal q learning (s,a), instead of action take option (s,o)

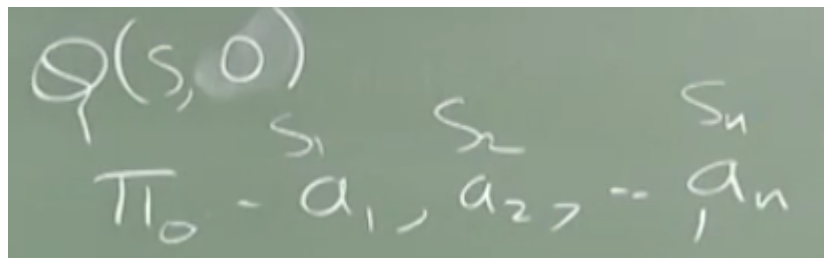
MDP + Options gives SMDP

Reward - since option started and option ended (r-t to r-t+tau)

Intra options Q-learning:

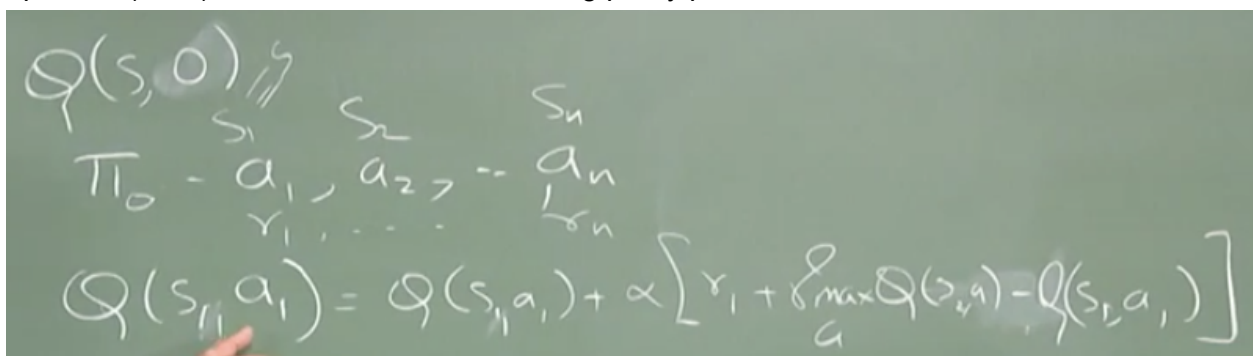


Options consists of primitive actions, update Q(s,o)



Actions I see: then terminate

Update Q(s, a1) - can't use a2 - i am following policy pi-not



After pi-not terminates, i will do what i want

I am not following π , but π -not. Don't know if will follow π -not in a_2 and claim it is sarsa

Is there anything else i can do, can i update any other value function

$$Q(s_1, a_2) = -$$

Sub-problems are solved, allowed to take primitive action

Intra-option Q-Lmg

$$Q(s, 0) \quad s_1 \quad s_2 \quad \dots \quad s_n$$

$$\pi_0 = a_1, a_2, \dots, a_n$$

$$r_1, \dots, r_n$$

$$Q(s_1, a_1) = Q(s_1, a_1) + \alpha [r_1 + \max_a Q(s_2, a) - Q(s_1, a_1)]$$

$$Q(s_1, a_2) = -$$

$$Q(s_2, 0) ?$$

How about $Q(s_2, 0)$? Can i update it when option is ended

Option has to be Markov - o started in s_1

I should not be using π -o, i can't use this policy

SMDP reward should come from r_2 (not from r_1)

Assuming I am doing Markov, is there efficient way to update Q

15min: again

Don't know if π_0 is max, i have chosen s_1 , o, i need to take action π -o -> except when i end it

Don't need to have running sum

Bootstrapping

$$\begin{aligned} Q(s_1, o) &= Q(s_1, o) + \alpha \left[r_1 + \gamma Q(s_2, o) - Q(s_1, o) \right] \text{ if not ending at } s_2 \\ &= Q(s_1, o) + \alpha \left[r_1 + \gamma \max_a Q(s_2, a) - Q(s_1, o) \right] \text{ if terminates at } s_2 \end{aligned}$$

Prob 1-beta will do 1st

With prob beta will do 2nd

$$\begin{aligned} Q(s_1, o) &= Q(s_1, o) + \alpha \left[r_1 + \gamma^{1-\beta(s_1)} Q(s_2, o) - Q(s_1, o) \right] \text{ if not ending at } s_2 \\ &= Q(s_1, o) + \alpha \left[r_1 + \gamma^{1-\beta(s_1)} \max_a Q(s_2, a) - Q(s_1, o) \right] \text{ if terminates at } s_2 \end{aligned}$$

What else can you do?

Suppose

$$\begin{aligned} \pi_o(s_1) &= a_1 \\ \pi_o(s_1) &= a_1 \end{aligned}$$

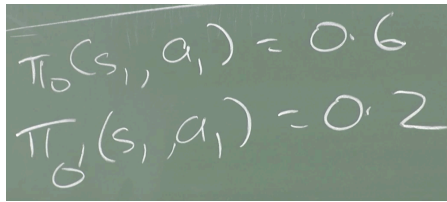
not according to time but

How can I update $Q(s, o')$ given that I have taken action o

Can do same thing - writing only first expression - s_1, a_1, r_1 (don't care π_o or $\pi_{o'}$)

$$Q(s, o') = Q(s, o') + \alpha \left[r_1 + \gamma Q(s_2, o') - Q(s, o') \right]$$

Update all options which are consistent with that option, that i am executing
Another question


$$\pi_0(s_1, a_1) = 0.6$$
$$\pi_{0'}(s_1, a_1) = 0.2$$

Even if the options are consistent - can use same experience

Even if they are not consistent - can use important sampling

Why do i need to be in options space, I just need two policies

1. Behaviour policy - π_0
2. Estimation policy - $\pi_{0'}$

We can do more generalisation

I can learn about all option - with off policy learning

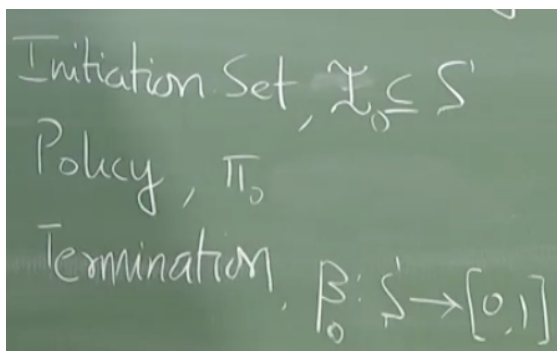
We have to come up with Options first - active area of Research

Research - Options Discovery (REF)

Because of simplicity -

Little dis-satisfying

1. Where $\pi_{0'}$ is defined



Initiation Set, $\mathcal{I}_0 \subseteq S$
Policy, π_0
Termination, $\beta_0: S \rightarrow [0,1]$

termination cuts from initiation sets

2. Will behave differently even if it is Markov

$$Q(s, a)$$
$$s_1 \quad s_2 \quad \dots \quad s_n$$
$$\pi_0 \quad a_1, a_2, \dots, a_n$$

For outside observer - i am behaving in non-markov or stochastic way

I may be executing deterministic policy

- In one sense it is power of it - you can do what you want

w10L59 HAM - Hierarchical Abstract Machines

Ref: Read first 4 sections: Recent advances in Hierarchical RL

- Andy, Sridhar
- Section5: Concatenated all his work !!! this is off the record! Skewed view
- What is recent from his lab in RL

2nd Framework; Hierarchical Abstract Machines

Reference: [R. Parr, S. Russell. 1997. Reinforcement Learning with Hierarchies of Machines](#)

It is most elegant and most complete

- People have not built significant structure
- Proposed by Ron Parr, Russel

Simple machines encode

Define - Hierarchy of FSMs

Four types of states:

1. Action state:
 - a. Take input emit action (only in this state emit action)
2. Call state:
 - a. Takes an input, call another machine
3. Choice state:
 - a. How will you encode decision making in FSM
 - b. Give non-determinism transition to two or more states
 - c. Can make call to call state, not just to primitive state
4. Stop state:
 - a. Stops and returns control back to machine that call it

There will be a Starting machine (top level M-not state)

- b. what is the input - underlying states of mdp
- c. Output is primitive action (input primitive states)

How hierarchy gets established here:

- Calls. Don't have loop. It is a proper graph - it is DAG

In some machine we will get primitive state

- From top down actions (go to reach station) to reach class where you take right

Get out of room machine called it

Go down the stair machine - series of steps
Walk out of the door machine
Go to the GC machine

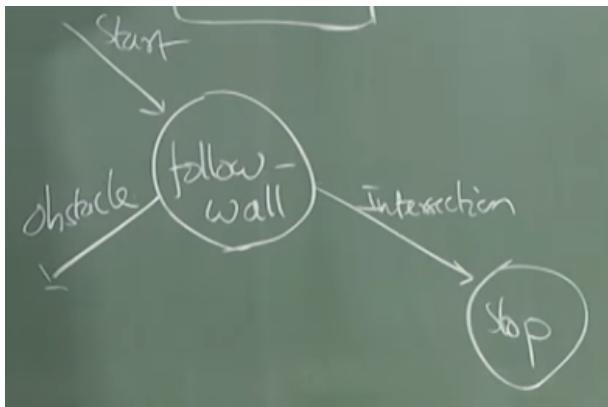
Choice state Go to the lift i

14min: complex grid world task



(right) follow wall from obstacle or come back and move right up

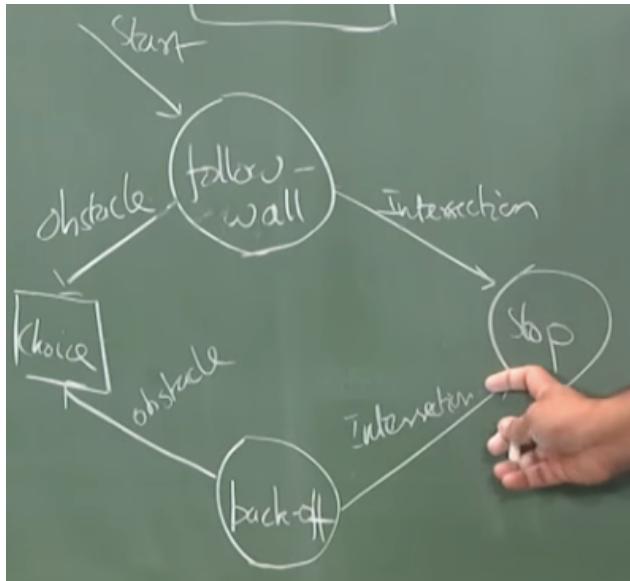
Follow wall - call state. When follow wall machine returns - i may be at intersection or at obstacle



Navigating corridor - just walk in corridor

At intersection - i am happy

At Obstacle have choice - pick diff wall and follow or backoff



this is illustrative machine - not real

Follow wall machine - will get action move right, left -> with Action State

Question

- where is RL - what do you learn?
- where is SMDP here?

What do you think I should be learning -> choice state to could be another action

Core (basic) MDP state - I need to learn **what Choice I make**

I need to have

- base mdp state
- what action i pick will depend on what machine i am in
- It also depends on who called it
- It is a call stack -> has to be part of state space

What are my actions (action space):

- I will ignore primitive action (these are learnt)
- Transition possible out of the state
- Once i call follow wall

Hi is call stack cross Si core mdp state = state space

Action space output of choice state

Mi - what states i am in

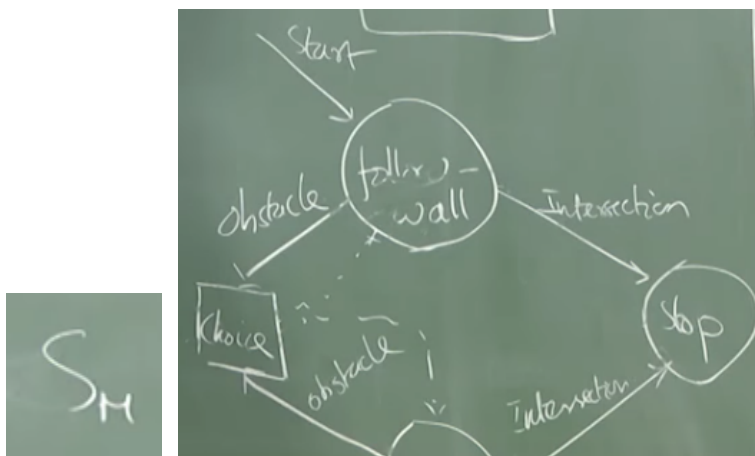
If no choice states - these will not appear in action space as nothing to learn

H_i - Call Stack,
 S_i - Core MDP state
 M_i - Machine State

In options - frozen the hierarchy
 recursively optimal = hierarchical optimal (can't change the policy)

Which machine

Actions are machine states S-M



where i am in machine

You can learning using SMDP Q-learning

Call action or Choice state - NO transition in Core mdp

Whenever i reach action state - core mdp state will change

What will be the equivalent of already learnt policy in this case

Any sequence of action state you state, without choice state in between = it is policy

- Already pre-packaged
- Little more complex than Options framework

Powerful - recursive, hierarchical optimality

Allows you to work in partially observed states (don't need to know full state space)

Can use FSM to define more complex policies

Can define non markov policies

FSMs with memory

Disadvantage: it is complex